Przygotowanie konta GitHub i środowiska do kursu

Dla osób, które mają już konto GitHub · ok. 30–40 minut · jednorazowo

Po tej instrukcji będziesz mieć: własną kopię (fork) repozytorium kursu, sklonowaną na komputer, działające środowisko Pythona i wiedzę, jak oddać ćwiczenie przez Pull Request. Repozytorium kursu:

`https://github.com/StanislawDrosio/DataSciencePythonExercises`

Uwaga: Wszystkie komendy wpisujesz w terminalu: macOS – aplikacja *Terminal*; Windows – *PowerShell* lub *Git Bash*. Jeśli komenda zwraca `command not found` / „nie rozpoznano”, narzędzie nie jest zainstalowane – wróć do odpowiedniego kroku.

Krok 1. Zainstaluj Git

macOS: wpisz `git --version`. Jeśli system zaproponuje instalację narzędzi deweloperskich, zgódź się (albo `brew install git`).
Windows: pobierz „Git for Windows” ze strony git-scm.com i zainstaluj z domyślnymi opcjami.

```
git --version
```

Sprawdź: pojawia się numer wersji, np. `git version 2.4x.x`.

Krok 2. Skonfiguruj swoje dane w Git

Git podpisuje commity imieniem i e-mailem. Użyj e-maila przypisanego do konta GitHub (lub adresu „noreply” z *Settings* → *Emails*, jeśli nie chcesz ujawniać prawdziwego):

```
git config --global user.name  "Imię Nazwisko"
git config --global user.email "twoj-email@example.com"
git config --global init.defaultBranch main
```

Krok 3. Zainstaluj GitHub CLI i zaloguj się

Najprostszy sposób na uwierzytelnienie w GitHubie (GitHub nie przyjmuje już haseł w terminalu – bez tego `git push` się nie uda).

```
# macOS:
brew install gh

# Windows (PowerShell):
winget install --id GitHub.cli

# potem (oba systemy; po instalacji otwórz terminal ponownie):
gh auth login
```

W kreatorze wybierz: **GitHub.com** → **HTTPS** → **Yes** (authenticate Git with your GitHub credentials) → **Login with a web browser**. Skopiuj kod z terminala, wklej go w przeglądarce i zaloguj się na **swoje** konto.

```
gh auth status
```

Sprawdź: widzisz „Logged in to github.com account TWÓJ_LOGIN”.

Krok 4. Zrób fork repozytorium kursu

Fork to Twoja własna kopia repozytorium na GitHubie. Zmiany robisz u siebie, a potem proponujesz je prowadzącemu przez Pull Request.

1. Otwórz `https://github.com/StanislawDrosio/DataSciencePythonExercises` w przeglądarce.
2. Kliknij **Fork** (prawy górny róg).
3. W polu *Owner* zostaw swoje konto, nazwę zostaw bez zmian i kliknij **Create fork**.

Sprawdź: w pasku adresu jest `github.com/TWÓJ_LOGIN/DataSciencePythonExercises`, a pod nazwą napis „forked from StanislawDrosio/…”.

Krok 5. Sklonuj swój fork na komputer

Wybierz katalog, w którym trzymasz projekty (np. Dokumenty), i wpisz (zastąp `TWÓJ_LOGIN`!):

```
git clone https://github.com/TWÓJ_LOGIN/DataSciencePythonExercises.git
cd DataSciencePythonExercises
```

Dodaj też repozytorium prowadzącego jako `upstream` – dzięki temu pobierzesz nowe laboratoria:

```
git remote add upstream https://github.com/StanislawDrosio/DataSciencePythonExercises.git
git remote -v
```

Sprawdź: widzisz dwa zdalne repozytoria: `origin` (Twój fork) i `upstream` (prowadzący).

Krok 6. Zainstaluj Pythona i środowisko

Wariant A – conda (polecany): zainstaluj Miniconda (docs.conda.io → Miniconda), otwórz nowy terminal i w katalogu repo:

```
conda env create -f environment.yml
conda activate ds-exercises
```

Wariant B – bez condy (potrzebny Python 3.11+):

```
# macOS/Linux:
python3 -m venv .venv && source .venv/bin/activate

# Windows PowerShell:
python -m venv .venv
.venv\Scripts\Activate.ps1

pip install -r requirements.txt
```

Sprawdź instalację:

```
python -c "import pandas, numpy, sklearn; print('OK', pandas.__version__)"
jupyter lab --version
```

Sprawdź: wypisuje „OK” z numerem wersji pandas oraz numer wersji JupyterLab.

Krok 7. Uruchom pierwszy test (powinien się wywalić!)

Sprawdzamy, że narzędzia działają. W katalogu repo wpisz:

```
pytest tests/test_lab01.py -v
```

Sprawdź: testy **nie przechodzą** (FAILED, `NotImplementedError: TODO`). To dobrze: szablon jest pusty, a pytest działa. Po rozwiązaniu labu zobaczysz zielone PASSED.

Krok 8. Przećwicz cały workflow na Lab 01

Każde laboratorium oddajesz tak samo. Podstawiasz swój login:

```
git checkout -b lab01-TWÓJ_LOGIN
mkdir -p submissions/TWÓJ_LOGIN
cp -r labs/lab01 submissions/TWÓJ_LOGIN/lab01        # Windows: Copy-Item -Recurse labs/lab01 submissio
jupyter lab submissions/TWÓJ_LOGIN/lab01/lab.ipynb
```

Rozwiąż zadania w `exercises.py` (w folderze *submissions*, nie w *labs*!). Testuj lokalnie:

```
# macOS/Linux:
SUBMISSION_DIR=submissions/TWÓJ_LOGIN pytest tests/test_lab01.py -v

# Windows PowerShell:
$env:SUBMISSION_DIR="submissions/TWÓJ_LOGIN"; pytest tests/test_lab01.py -v
```

Gdy testy są zielone, wyślij pracę na GitHuba:

```
git add submissions/TWÓJ_LOGIN/lab01
git commit -m "Lab 01: slugify, word_freq"
git push -u origin lab01-TWÓJ_LOGIN
```

Krok 9. Otwórz Pull Request

Po `git push` w terminalu pojawi się link – kliknij go. Albo wejdź na swój fork w przeglądarce i kliknij **Compare & pull request**. Sprawdź, że: *base repository* = `StanislawDrosio/DataSciencePythonExercises`, *base* = `main`. Wypełnij szablon (w tym **deklarację użycia AI**) i kliknij **Create pull request**. Alternatywnie z terminala: `gh pr create`.

Sprawdź: w zakładce *Checks* PR-a uruchamia się „autograde”. Zielony ✓ = testy przeszły. Przy pierwszym PR może być napis „**Awaiting approval**” – to normalne, prowadzący musi raz zatwierdzić uruchomienie.

Uwaga: PR zmienia pliki **tylko** w `submissions/TWÓJ_LOGIN/`. Zmiana czegokolwiek innego (np. w *labs* czy *tests*) sprawi, że autograder odrzuci PR. Nie klikaj „Merge” – zrobi to prowadzący (lub zamknie PR po ocenie).

Krok 10. Poprawki i kolejne laboratoria

Poprawka w tym samym PR: edytujesz pliki, potem `git add`, `git commit`, `git push` – PR odświeża się sam.
Nowe laboratorium: najpierw pobierz nowości od prowadzącego, potem nowa gałąź od aktualnego *main*:

```
git checkout main
git pull upstream main
git push origin main        # aktualizuje Twój fork
git checkout -b lab02-TWÓJ_LOGIN
```

Lista kontrolna – wszystko gotowe, gdy…

☐	Masz konto GitHub, a prowadzący zna Twój login
☐	`git --version` i `gh auth status` działają
☐	Masz fork i sklonowane repo z remote'ami *origin* + *upstream*
☐	Środowisko działa (import pandas/sklearn, jupyter lab)
☐	`pytest tests/test_lab01.py` uruchamia się (na szablonie – z błędami)
☐	Rozumiesz: praca w *submissions/login/labNN*, gałąź *labNN-login*, PR do *main* prowadzącego

Najczęstsze problemy

Objaw	Rozwiązanie
`Authentication failed` / prośba o hasło przy `git push`	Hasła nie działają. Wykonaj `gh auth login` (krok 3). Login to nazwa użytkownika GitHub, nie e-mail.
`remote: Permission denied` / 403 przy pushu	Pushujesz do repozytorium prowadzącego zamiast do forka. Sprawdź `git remote -v`: *origin* musi wskazywać na **Twój** login.
`command not found: pytest` / `jupyter`	Nie aktywowałeś środowiska. Wykonaj `conda activate ds-exercises` (lub aktywację *.venv*) i spróbuj ponownie.
Test: `FileNotFoundError ... exercises.py`	Brak zmiennej `SUBMISSION_DIR` albo skopiowanego folderu. Sprawdź, czy istnieje *submissions/login/labNN/exercises.py*.
Autograder: „Plik poza submissions/…”	W PR jest zmiana poza Twoim folderem. Cofnij ją: `git checkout main -- ścieżka`, zrób commit i push.
Autograder: login się nie zgadza	Nazwa folderu w *submissions* musi być dokładnie Twoim loginem GitHub (wielkość liter nie ma znaczenia).
Konflikty przy `git pull upstream main`	Nie edytuj plików w *labs/* ani *tests/* – wtedy konfliktów nie ma. Jeśli się pojawią, napisz do prowadzącego.
PR bez „Checks”, napis „Awaiting approval”	Poczekaj na zatwierdzenie przez prowadzącego (zwykle jednorazowo).

Pomoc: dokumentacja docs.github.com (Fork a repository, Set up Git) · pytaj na zajęciach lub w kanale kursu. Gdy zgłaszasz błąd, wklej **pełny tekst komunikatu**, nie zdjęcie fragmentu.